

ACF – Application Component Framework

Zusammenfassung der architektonischen Änderungen
November 2025 – Mai 2026

Erstellt für:	Witold Gantzke, Software-Architekt
Repositories:	ImagingTools/Acf, ImagingTools/AcfSln
Zeitraum:	November 2025 – Mai 2026
Datum:	10. Mai 2026
Versionen:	v1.0.0 (Jan 2026) → HEAD (Mai 2026)

Inhaltsverzeichnis

1. Übersicht der Änderungen
2. Smart-Pointer-Architektur (istd)
3. Neue Schnittstelle: IParamsInfoProvider (iprm)
4. Observer-Pattern-Refactoring (ibase)
5. TContainer – STL-Kompatibilität (ibase)
6. JSON-Serialisierung (iser/ifile)
7. Fortschrittsdialog: CSubtaskProgressDialog (iqtgui)
8. IDocumentMetaInfo – Typsichere Metadaten (idoc)
9. Komponentenumgebungs-Manager (icomp)
10. Logging: UseMainThread (ilog)
11. IDeviceBasedPersistence – Enum-basierter Status (ifile)
12. Weitere Änderungen
13. Compositor – Änderungen (AcfSIn)

1. Übersicht der Änderungen

Im Zeitraum November 2025 bis Mai 2026 wurde das ACF-Framework in mehreren Bereichen architektonisch weiterentwickelt. Die Änderungen umfassen sowohl grundlegende Infrastruktur-Verbesserungen (Smart-Pointer, Serialisierung, Observer-Pattern) als auch neue Schnittstellen und GUI-Komponenten. Parallel dazu wurde die Compositor-Anwendung im AcfSIn-Repository signifikant erweitert.

Änderungsübersicht nach Bibliothek

Bibliothek	Komponente	Art der Änderung	Auswirkung
istd	TInterfacePtr / TSharedInterfacePtr TUniqueInterfacePtr	Erweiterung	Hoch – Grundlegende Pointer-Semantik
iprm	IParamsInfoProvider	Neue Schnittstelle	Mittel – Parameter-Metadaten
ibase	CListObserver / CListObserverDispatcher	Refactoring	Mittel – Observer-Pattern
ibase	TContainer	Erweiterung	Mittel – STL-Kompatibilität
iser/ifile	CJsonMem/FileRead/WriteArchive	Bugfixes + Erweiterung	Hoch – Serialisierung
iqtgui	CSubtaskProgressDialog	Neue Klasse	Niedrig – UI-Komponente
idoc	IDocumentMetaInfo	Erweiterung	Mittel – Typsichere Metadaten
icomp	IComponentEnvironmentManager	Erweiterung	Mittel – Projektziele
ilog	CLogCompBase	Erweiterung	Niedrig – Thread-Steuerung
ifile	IDeviceBasedPersistence	Refactoring	Mittel – Enum-Status
iwidgets	CCurrentPageStackedWidget	Neue Klasse	Niedrig – Layout
icmm	CFastColor	Erweiterung	Niedrig – C++20 Konstruktor

Mitwirkende

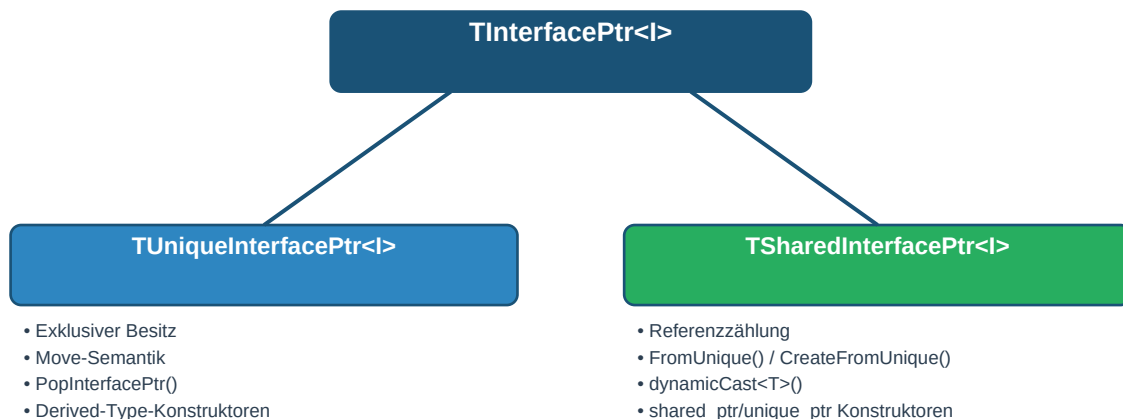
Entwickler	Hauptbeiträge
Oleksii Maistrenko	TInterfacePtr, CListObserver, TContainer, IDeviceBasedPersistence, CFastColor
Kirill Lepskiy	IComponentEnvironmentManager, Bitmap-Fixes, Compositor-Architektur
Vladimir Cherepanov	IParamsInfoProvider, TSimComponentsFactory, icmm
Roman Kucherenko	JSON-Serialisierung (Bugfixes + Optimierung)
Sergey Zastrovnyh	IDocumentMetaInfo (typsichere Metadaten)
Sascha Tutus	Params-Manager Crash-Fix, Pointer-Bug-Fix

Entwickler	Hauptbeiträge
Leonel Mora	CCurrentPageStackedWidget, StackWidget-Sizing
Alexander Naskin	Conan-Integration, Versionsverwaltung
GitHub Copilot	CSubtaskProgressDialog, CLogCompBase, JSON-Bugfixes, Compositor-Features

2. Smart-Pointer-Architektur (istd)

Die Smart-Pointer-Infrastruktur in `istd` wurde grundlegend modernisiert. Die Klassen `TInterfacePtr`, `TUniqueInterfacePtr` und `TSharedInterfacePtr` bilden nun eine konsistente Hierarchie mit separater Verwaltung von Root-Objekt und Interface-Pointer.

Smart-Pointer Typenhierarchie (istd)



Neue API-Methoden

Klasse	Neue Methode / Feature	Beschreibung
<code>TInterfacePtr</code>	<code>get()</code>	STL-kompatible Zugriffsmethode (wie <code>QSharedPointer::get()</code>)
<code>TInterfacePtr</code>	<code>operator==/!=(nullptr)</code>	Direkte <code>nullptr</code> -Vergleiche
<code>TInterfacePtr</code>	Konstruktor von abgeleiteten Typen	Implizite Aufwärtskonvertierung (Upcast)
<code>TUniqueInterfacePtr</code>	Move-Konstruktor	Move-Semantik für abgeleitete Typen
<code>TUniqueInterfacePtr</code>	<code>PopInterfacePtr()</code>	Intelligente Interface-Extraktion (3 Fälle)
<code>TUniqueInterfacePtr</code>	<code>MoveCastedPtr()</code>	Typ-validierter Move mit <code>dynamic_cast</code>
<code>TSharedInterfacePtr</code>	<code>FromUnique()</code>	Sichere Übertragung <code>Unique</code> → <code>Shared</code>
<code>TSharedInterfacePtr</code>	<code>CreateFromUnique()</code>	Statische Factory mit <code>dynamic_cast</code>
<code>TSharedInterfacePtr</code>	<code>dynamicCast()</code>	Qt-ähnliche Template-Cast-Methode
<code>TSharedInterfacePtr</code>	Konstruktor aus <code>shared_ptr/unique_ptr</code>	STL-Interoperabilität
<code>TSharedInterfacePtr</code>	Deprecation non-rvalue Konstruktoren	Erzwingung der Move-Semantik

Architektonische Motivation: Die Trennung von Root-Objekt und Interface-Pointer ermöglicht polymorphe Komponenten, bei denen das Root-Objekt (z.B. `CComponentBase`) mehrere Interfaces implementiert. Die Move-Semantik und Derived-Type-Konstruktoren reduzieren unnötige Kopien und

vereinfachen den Code bei Up-/Downcasts.

Datei: Include/istd/TInterfacePtr.h

3. Neue Schnittstelle: IParamsInfoProvider (iprm)

Die neue Schnittstelle IParamsInfoProvider ermöglicht den Zugriff auf Metadaten einzelner Parameter. Sie erbt von `istd::IChangeable` und liefert strukturierte Informationen über Parameter-Name und -Beschreibung.

```
struct ParamInfo {
    QString name;
    QString description;
};

virtual std::unique_ptr<ParamInfo> GetParamInfo(
    const QByteArray& paramId) const = 0;
```

Integration: Die Schnittstelle wurde in allen IParamsSet-Implementierungen eingebaut, einschließlich CComposedParamsSetComp. Die Rückgabe als `std::unique_ptr` signalisiert klare Besitzverhältnisse und erlaubt `nullptr` als "nicht gefunden"-Antwort.

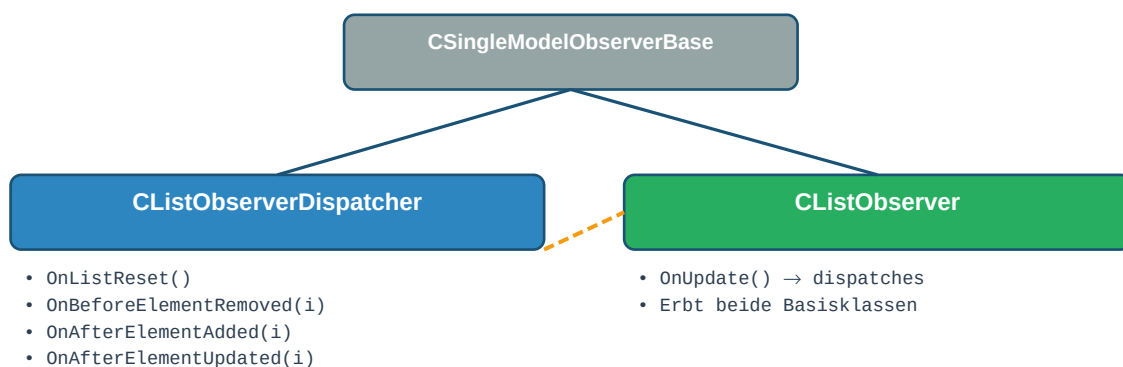
Dateien:

- Include/iprm/IParamsInfoProvider.h
- Tests/ParamsInfoProviderTest/

4. Observer-Pattern-Refactoring (ibase)

Das Observer-Pattern für Listenänderungen wurde in zwei separate Klassen aufgeteilt: CListObserverDispatcher (reines Dispatch-Verhalten) und CListObserver (konkreter Observer mit Model-Binding).

Observer-Pattern: Dispatcher/Observer-Trennung



Vorteile der Trennung:

- **CListObserverDispatcher** kann als Mixin verwendet werden, ohne CSingleModelObserverBase zu erben
- Bessere Separation of Concerns: Dispatch-Logik vs. Model-Binding
- Reset-Event überspringt nachfolgende Änderungen (Optimierung)
- Forward-Benachrichtigungen an Subelemente bei Dokumentlisten-Änderungen

Datei: Include/ibase/CListObserver.h

5. TContainer – STL-Kompatibilität (ibase)

Die Template-Klasse `TContainer<ItemClass, ContainerType>` wurde um STL-kompatible Methoden erweitert, um die Interoperabilität mit Qt- und Standard-Algorithmen zu verbessern.

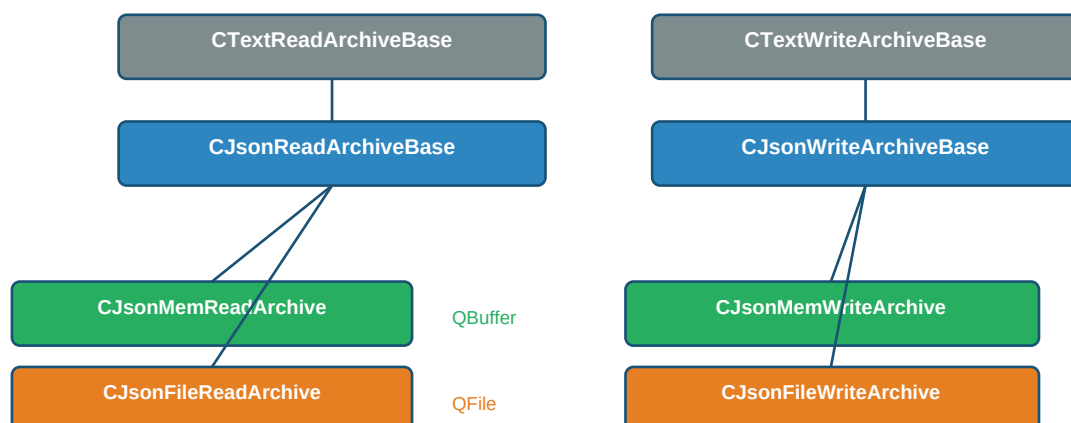
Neue Methode	Signatur	Beschreibung
insert (copy)	insert(qsizetype, const ItemClass&)	Einfügen per Kopie
insert (move)	insert(qsizetype, ItemClass&&)	Einfügen per Move
remove	remove(qsizetype)	Entfernen nach Index
resize	resize(qsizetype)	Größe ändern
clear	clear()	Alle Elemente entfernen
size	size() const	Elementanzahl
begin/end	begin(), end(), cbegin(), cend()	Iterator-Unterstützung
operator==	operator==(const TContainer&)	Vergleich
value_type	typedef ItemClass value_type	STL-Container-Trait

Datei: Include/ibase/TContainer.h

6. JSON-Serialisierung (iser/ifile)

Die JSON-Serialisierungsbibliothek wurde um speicher- und dateibasierte Varianten erweitert und mehrere Korrektheits-Probleme wurden behoben (Root-Tag-Handling, Escaping, Iterator-State-Guards).

JSON-Serialisierungsarchitektur



Klasse	I/O-Device	Hauptverwendung
CJsonMemReadArchive	QBuffer (In-Memory)	Deserialisierung aus QByteArray
CJsonMemWriteArchive	QBuffer (In-Memory)	Serialisierung in QByteArray (GetData())

Klasse	I/O-Device	Hauptverwendung
CJsonFileReadArchive	QFile	Deserialisierung aus Datei
CJsonFileWriteArchive	QFile	Serialisierung in Datei

Bugfixes:

- Root-Tag-Regression behoben
- JSON-Escaping korrigiert
- Iterator-State-Guard für leere Reader hinzugefügt
- Regressionstests erweitert

Dateien:

- Include/iser/CJsonMemReadArchive.h
- Include/iser/CJsonMemWriteArchive.h
- Include/ifile/CJsonFileReadArchive.h
- Include/ifile/CJsonFileWriteArchive.h

7. Fortschrittsdialog: CSubtaskProgressDialog (iqtgui)

Die neue Klasse CSubtaskProgressDialog erweitert QDialog und CCumulatedProgressManagerBase, um einen mehrstufigen Fortschrittsdialog mit einzelnen Subtask-Anzeigen bereitzustellen.

Feature	Beschreibung
Gesamtfortschritt	Kumulierter Fortschrittsbalken über alle Subtasks
Subtask-Widgets	Dynamische Widgets mit Status (✓/→/■), Beschreibung, Balken
Thread-Sicherheit	Qt-Signale für sichere GUI-Updates aus Worker-Threads
Auto-Ausblendung	Subtask-Bereich wird bei nur einem Task ausgeblendet

Datei: Include/iqtgui/CSubtaskProgressDialog.h

8. IDocumentMetaInfo – Typsichere Metadaten (idoc)

Die Schnittstelle IDocumentMetaInfo wurde um eine typsichere Template-Methode erweitert, die std::optional<T> für die Rückgabe nutzt und eine Laufzeit-Typprüfung durchführt.

```
template<class T>
static std::optional<T> GetMetaInfoT(
    const idoc::IDocumentMetaInfo& metaInfo, int key);
```

Verhalten: Prüft den QVariant-Typ zur Laufzeit und gibt std::nullopt bei Typ-Mismatch zurück (mit Debug-Warnung). Kompatibel mit Qt 5 (userType()) und Qt 6 (typeId()).

Datei: Include/idoc/IDocumentMetaInfo.h

9. Komponentenumgebungs-Manager (icomp)

Die Schnittstelle IComponentEnvironmentManager wurde um die Methode GetProjectTargets() erweitert, die eine Liste der Projektzielen (Dateipfade zu Root-Registries der in der Umgebungsconfiguration definierten Executables) zurückgibt.

```
virtual QStringList GetProjectTargets() const = 0;
```

Verwendung: Diese Methode wird vom Compositor für die Export-Auflösung verwendet, um Root-Registries zu identifizieren und die Komponentenhierarchie vollständig zu traversieren.

Datei: Include/icomp/IComponentEnvironmentManager.h

10. Logging: UseMainThread (ilog)

Die Basisklasse CLogCompBase erhielt ein neues konfigurierbares Attribut UseMainThread, das steuert, ob Log-Nachrichten aus Worker-Threads über die Qt-Event-Queue an den Haupt-Thread weitergeleitet

werden.

UseMainThread	Verhalten	Anwendungsfall
true (Standard)	Asynchron via Qt-Signal/Slot	GUI-Logging, nicht thread-sichere Handler
false	Synchron im aufrufenden Thread	Thread-sichere Handler, minimale Latenz

Datei: Include/i log/CLogCompBase.h

11. IDeviceBasedPersistence – Enum-basierter Status (ifile)

Die Rückgabewerte der Methoden `ReadFromDevice()` und `WriteToDevice()` wurden von `bool` auf eine Enum-basierte Zustandsanzeige umgestellt.

```
enum OperationState {
    Successful, // Operation erfolgreich
    Canceled,   // Vom Benutzer abgebrochen
    Failed      // Operation fehlgeschlagen
};
```

Motivation: Die Unterscheidung zwischen "Abgebrochen" und "Fehlgeschlagen" ermöglicht eine differenziertere Fehlerbehandlung in der Benutzeroberfläche und verhindert unnötige Fehlermeldungen bei benutzergesteuerten Abbrüchen.

Datei: `Include/ifile/IDeviceBasedPersistence.h`

12. Weitere Änderungen

CCurrentPageStackedWidget (iwidgets)

Neues Widget, das `QStackedWidget` erweitert und dessen `sizeHint()` und `minimumSizeHint()` an den Inhalt der aktiven Seite anpasst. Löst Layout-Probleme bei dynamisch wechselnden Seiteninhalten.

CFastColor – Iterator-Paar-Konstruktor (icmm)

C++20-basierter Konstruktor mit `std::input_iterator` Concept-Requirement. Ermöglicht Range-basierte Initialisierung: `CFastColor(vec.begin(), vec.end())`.

TSimComponentsFactory-Fix (icomp)

Korrektur der Template-Komponentenfabrik für korrekte Instanziierung.

Conan-Integration

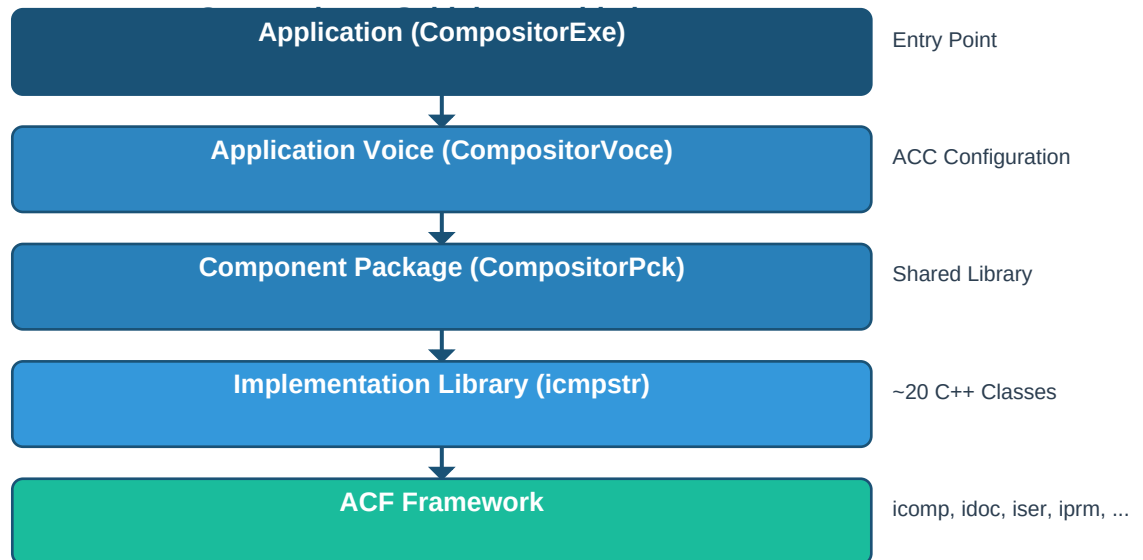
Neue `conanfile.py` für Package-Management mit automatischer Versionsgenerierung aus dem Build-System.

Iterator-Invalidierungs-Crash (iview)

Fix für Crash bei `DeselectAllShapes()` während Maus-Events durch sichere Iterator-Verwaltung.

13. Compositor – Änderungen (AcfSIn)

Der Compositor ist ein visueller Editor für ACF-Komponentenkonfigurationen (*.acc-Dateien). Im Berichtszeitraum wurde er signifikant erweitert, insbesondere um Navigations- und Synchronisierungsfunktionen.



Neue Features und Verbesserungen

Feature	PR(s)	Beschreibung
Component Tree Dock-Fenster	#30, #31, #33	Hierarchische Baumansicht aller Komponenten mit Root-Registry-Auswahl, Textfilter und Doppelklick-Navigation zum Diagramm
Packages & Tree kombiniertes Dock	#33	Integration von Package-Browser und Component Tree in ein einzelnes Tab-basiertes Dock-Widget mittels ComposedGui
Export-Auflösung und Navigation	#27	Tracking delegierter Attribute über IRegistry-Interfaces: Bottom-Up-Auflösung, Visited-Set statt chainDepth, BFS für Export-Ketten, automatische Attribut-Selektion
Selektions-synchronisierung	#31, #34, #36, #38, #41	Bidirektionale Synchronisierung zwischen Diagramm und Component Tree: Selektion, Deselektion, Bold-Hervorhebung des aktiven Dokuments
Worker-Thread Document Processing	#22, #23	Auslagerung von DoDocumentProcessing in Worker-Thread (CDocumentProcessingWorkerThread), UI-Operationen bleiben im Haupt-Thread
Fortschritts-verwaltung	#20, #21	Erweiterte und flexiblere Fortschrittsverwaltung, Fix für Progress-Reset in DoProcessingToOutput

Feature	PR(s)	Beschreibung
Persistente Root-Registry-Auswahl	#37	Root-Registry-Auswahl wird über TRestorableGuiWrap in QSettings gespeichert und beim Neustart wiederhergestellt (deferred restore)
Multi-Selection Drag-Fix	#29	Korrektur des Drag-Verhaltens bei Mehrfachselektion im Komponentendiagramm
Komponentenumbenennung Selektion-Fix	#38, #41	Beibehaltung der Selektion nach Umbenennung mittels CChangeGroup und Selection-ID-Update vor RenameElement
Dock-Widget Breite	#32	Vergrößerung der linken und rechten Dock-Widgets um 100px
Dokumentation	#26	README.md für Compositor-Applikation erstellt

Architektur: Export-Auflösung

Die Export-Auflösung ist ein zentrales neues Feature, das die Navigation in komplexen Komponentenhierarchien ermöglicht. Der Algorithmus funktioniert wie folgt:

- 1. Root-Registry-Identifikation:** Über `GetProjectTargets()` werden alle Root-Registries identifiziert (leere PackageId).
- 2. Bottom-Up-Traversierung:** Vom aktuellen Attribut wird die Export-Kette aufwärts verfolgt.
- 3. Visited-Set:** Ein `QSet<exportId>` verhindert exponentielles Wachstum statt einfacher chainDepth-Begrenzung.
- 4. BFS-Suche:** Breitensuche bevorzugt Auflösungen auf der aktuellen Hierarchieebene.
- 5. Disambiguierung:** Bei mehreren Auflösungskandidaten wird ein Dialog angezeigt, der die Root-Registry vorselektiert.
- 6. Navigation:** Nach der Auflösung wird das Zieldokument geöffnet, das Element selektiert und das Attribut in der Attribut-Editor-Ansicht hervorgehoben.

Architektur: Worker-Thread Document Processing

Die Dokumentverarbeitung wurde in zwei Phasen aufgeteilt:

Phase	Thread	Aufgaben
Vor-/Nachbereitung	UI-Thread (Main)	Parameter-Setup, Progress-UI-Updates, Ergebnis-Anzeige
Verarbeitung	Worker-Thread	<code>IProcessor::DoProcessing()</code> – rechenintensive Operationen

Die Klasse `CDocumentProcessingWorkerThread` kapselt den Worker-Thread und verwaltet Input/Output über `IPolymorphic-Interfaces`. Die Verarbeitungszeit wird gemessen und das Ergebnis als `ResultCode` zurückgegeben.

Dieses Dokument wurde automatisch aus den Git-Historien der Repositories ImagingTools/Acf und ImagingTools/AcfSIn generiert.